Nom de l'établissement

Département / Filière

Module : DevOps

Flashcards API

Une petite API, toute la chaîne DevOps

CI/CD · Docker · Kubernetes · Observabilité · Sécurité

Rapport final de projet



Réalisé par : Mouhib Fahem

Encadré par : Nom de l'enseignant(e)

Dépôt : `https://github.com/mouhibfahem/devops`

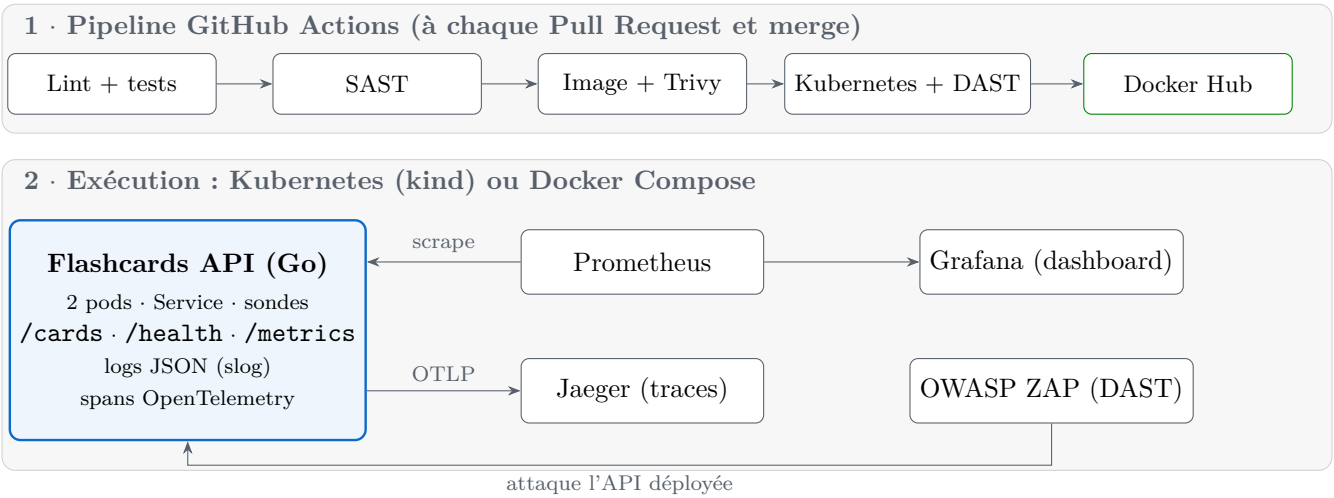
Image Docker : `https://hub.docker.com/r/mouhibfahem/flashcards-api`

Année universitaire 2026 – 2027 · Octobre 2026

1. Objectif et application

L'objectif était d'appliquer toute la chaîne DevOps à un petit service : du code jusqu'au déploiement, avec automatisation, observabilité et sécurité. L'application est une **API REST de flashcards** écrite en Go (bibliothèque standard, **149 lignes** dans `main.go`) qui aide à réviser des notions du cours. Elle permet de créer, lister, consulter et supprimer des cartes, et propose un **mode quiz** (`GET /cards/random`) qui renvoie une question au hasard sans sa réponse. Les tests unitaires couvrent **90,8 %** du code.

2. Architecture



Chaque tâche suit le même flux : une **issue** (10 issues, tableau Kanban public GitHub Projects), une **branche**, une **Pull Request** liée à l'issue, la vérification par le pipeline, puis le merge. Au total, **8 Pull Requests** ont été fusionnées.

3. Pipeline CI/CD (GitHub Actions)

Job	Ce qu'il fait
Lint, test, build	`gofmt`, `go vet`, tests avec détecteur de concurrence et couverture, compilation
SAST	gosec (résultats dans GitHub code scanning), govulncheck (dépendances), gitleaks (secrets dans l'historique)
Image	Build Docker unique puis scan **Trivy** : échec si une faille HIGH/CRITICAL corrigible est présente
Deploy + DAST	Cluster **kind** neuf, déploiement de l'image scannée, tests à travers le Service, scan **OWASP ZAP** de l'API déployée
Publish (main)	Push sur Docker Hub (`latest`, `sha-<commit>`) **uniquement si tout est vert**

Les identifiants Docker Hub sont des **secrets GitHub**, jamais dans le code. Les rapports (couverture, SAST, Trivy, ZAP) sont conservés comme artefacts de chaque exécution.

4. Conteneurisation

`Dockerfile` en deux étapes : `golang:1.27.2-alpine` compile un binaire statique, puis seul ce binaire est copié dans une image **distroless** (environ 30 Mo, sans shell, utilisateur non-root 65532). Le fichier `docker-compose.yml` lance l'API avec Prometheus, Grafana et Jaeger en une commande.

5. Observabilité

— **Logs** : une ligne JSON par requête (`log/slog`) avec méthode, route, statut, durée, `request_id`, `trace_id`. Le niveau suit le statut : INFO, WARN (4xx), ERROR (5xx et *panics* récupérées).
— **Métriques** : `/metrics` au format Prometheus : compteur de requêtes, histogramme des temps de réponse, requêtes en cours. Le label `route` utilise le motif (`GET /cards/{id}`) pour limiter le nombre de séries.
— **Dashboard** Grafana provisionné automatiquement : requêtes/s, taux d'erreurs, latence p50/p95/p99.
— **Tracing** : un span OpenTelemetry par requête, exporté en OTLP vers Jaeger ; un en-tête `traceparent` entrant est poursuivi.

Lien avec le cours SRE. La latence p95 et le taux d'erreurs sont des **SLI**. Un **SLO** possible : « 95 % des requêtes en moins de 200 ms et moins de 0,1 % d'erreurs 5xx sur un mois ». Lors des tests de charge, la p95 mesurée était inférieure à 1 ms.

6. Sécurité

Problème trouvé	Outil	Correction
11 vulnérabilités connues dans Go 1.27.0 et `x/net` v0.59.0	govulncheck	Toolchain go1.27.2 et `x/net` v0.60.0
Injection dans les logs (G706)	gosec	Attribut structuré `slog`
En-têtes `X-Content-Type-Options` et `Cross-Origin-Resource-Policy` absents	OWASP ZAP	En-têtes de sécurité sur toutes les réponses ; le second scan ne garde que des alertes informatives

Autres mesures : aucune fuite de secret (gitleaks), 0 faille HIGH/CRITICAL dans l'image (Trivy), corps de requête limité à 1 Mo, délais d'expiration serveur, conteneur non-root et en lecture seule.

7. Kubernetes

Les manifests `k8s/`, appliqués avec Kustomize, créent un namespace, une ConfigMap, un **Deployment de 2 réplicas** et un **Service** ClusterIP. Le Deployment définit des sondes *readiness* et *liveness* sur `/health`, des requêtes et limites CPU/mémoire, une mise à jour progressive sans interruption (`maxUnavailable: 0`) et un contexte de sécurité strict (non-root, système de fichiers en lecture seule, aucune capability Linux). Test en local avec kind : quand on supprime un pod, Kubernetes le recrée en quelques secondes.

8. Leçons apprises

— **Les scans trouvent de vrais problèmes.** govulncheck a révélé des failles dans la version même de Go ; sans automatisation, elles seraient passées inaperçues.
— **L'outillage évolue plus vite que les outils.** gosec ne lisait pas encore Go 1.27 et a dû être exécuté avec Go 1.26. Il faut vérifier qu'un outil analyse réellement tout le code.
— **Kubernetes est strict.** `runAsNonRoot` exige un UID numérique dans l'image ; l'erreur `CreateContainer ConfigError` l'a révélé.
— **L'état en mémoire empêche de passer à l'échelle.** Avec 2 réplicas, chaque pod a ses propres cartes ; une base de données serait l'étape suivante.
— **Petits incréments.** Une issue = une PR vérifiée par le pipeline : chaque erreur est restée petite et facile à corriger.

9. Pistes d'amélioration

Base de données (PostgreSQL) pour partager les cartes entre pods, déploiement sur un cloud (bonus), Prometheus et Grafana dans le cluster, alertes basées sur les SLO, et spans enfants pour un tracing plus détaillé.